

Introduction: Algorithmic notation –Programming principles –Creating programs
Analyzing programs. Arrays: One dimensional array, multidimensional array, pointer
arrays. Searching: Linear search, Binary Search, Fibonacci search. Sorting techniques:
Internal sorting - Insertion Sort, Selection Sort, Shell Sort, Bubble Sort, Quick Sort, Heap
Sort, Merge Sort and Radix Sort

Data structures, Algorithmic notation, Arrays

1. Define Abstract Data Type (Nov 2018, April 2017))

Abstract Data type (ADT) is a type (or class) for objects whose behaviour is defined by a set of value and a set of operations.

The definition of ADT only mentions what operations are to be performed but not how these operations will be implemented

The process of providing only the essentials and hiding the details is known as abstraction.

2 .List out the performance measure of an algorithm (Nov 2018)

Algorithm Efficiency

Time efficiency - a measure of amount of time for an algorithm to execute.

Space efficiency - a measure of the amount of memory needed for an algorithm to execute.

3 .Define data structure (Nov 2019, April 2017)

- It is defined as representation of data or storing or organization of data.
- Simply it is the combination of Data structure & algorithm.
- Data structure is the structural representation of logical relationships between elements of data.
- Data structure mainly specifies the structured organization of data, by providing accessing methods with correct degree of associatively.

4. What is one dimensional array? Example (Nov 2019)

An array is a fixed size sequence collection of elements of the same data type.

It is simply a grouping of like type data.

In its simplest form an array can be used to represent a list of numbers or a list of names.

There are two types of arrays:

1. One dimensional array.
2. Multi dimensional array.

The syntax for declaring the array is:

DATATYPE ARRAYNAME [SIZE]; Where ARRAY NAME = name of the array, SIZE = size of the array

Eg: int a[10];

Where a- name of the array

10- size of the array.

5. What is Algorithm (Dec 2016)

- Algorithm is defined as set of instruction to perform a particular task. In Other words algorithm is a step by step procedure to perform a specific task.

6. What is time Complexity (Dec 2016, Nov 2014?)

Time complexity of an algorithm is the amount of time (or number of steps) needed by a program to complete its task (i.e. to execute a particular algorithm). The way in which the number of steps required by an algorithm varies with the size of the problem it is solving. The time taken for algorithm is comprised of two times

- COMPILATION TIME
- RUN TIME

SORTING TECHNIQUES

7. Average case and worst case of Bubble sort?

Bubble sort has a worst-case and average complexity of $O(n^2)$, where n is the number of items being sorted. Most practical sorting algorithms have substantially better worst-case or average complexity, often $O(n \log n)$ When the list is already sorted (best-case), the complexity of bubble sort is only $O(n)$.

8. What are the basic operations of data structure?

There are different types of operations that can be performed for the manipulation of data in every data structure.

1. Traversing
2. Searching
3. Sorting
4. Insertion
5. Deletion.

9. What is mean by linear and non linear data structure? Give some Example (Nov 2017)

Linear data structure is data structure having a linear relationship between its adjacent elements. Linked lists are examples of linear data structure.

Non-linear data structures are data structures that don't have a linear relationship between its adjacent elements but have a hierarchical relationship between the elements. Trees and graphs are examples of nonlinear data structures.

10. State some of the operations that can be performed on a Array (Nov 2017)**OPERATION ON ARRAYS:**

- Sorting in ascending/descending order
- Searching an element in an array
- Deleting/Inserting an element in an array
- Finding out the largest or smallest element in an array
- Removing a duplicate (repeated elements) from the array.

11. State the various algorithmic notations (Nov 2015)

1. Big-Oh Notation(O)
2. Big-Omega Notation(Ω)
3. Big-Theta Notation(θ)
4. Little-oh Notation(o)
5. Little -omega Notation(ω)

12. What is an Array?(Nov 2015)

- An array is a finite ordered and collection of homogenous data elements. All the elements of an array are of the same data type only so it was termed as collection of homogenous elements.
- An array is a kind of linear data structure because all the elements of the array are stored in linear order.

13. Arrange the following Big -oh notation terms in asymptotic ascending order. 2^n , $\log n$, n , n^2 , $n \log n$.(Sep 2020).

$\log n$, $n \log n$, n , n^3 , 2^n , n^2 .

14. Convert the following from infix to postfix expression.
 $A+B*CA+B*C^D^E/(E+F)*G/H+K$.(Sep 2020).

$A B C A * + B C D E ^ ^ * E F + / G * H / + K +$

11 Marks

1. State the best case and worst case analysis of binary search (Nov 2017, nov 2015, Dec 2016, Nov 2014)

BINARY SEARCH

- The main objective of binary search is to search a particular element which is present in the list of elements. The list of elements should be in a increasing or non-decreasing or sorted order.
 - For searching lists with more values linear search is insufficient, binary search helps in searching larger lists. To search a particular item the approximate middle entry of the table is located, and its key value is examined.
1. If the search value is higher than the middle value, then the search is made with the elements after the middle element.
 2. If the search value is smaller than the middle element, then the search is made with the elements before the middle value.
 3. This process continues till the required target is found

Is $15 == A[3]$? No

$15 < A[3]$, repeat the steps with $low = 0$ and $high = mid - 1 = 2$

Is $low > high$? NO

$$Mid = (0 + 2) / 2 = 1.$$

8	10	15
[0]	[1]	[2]
↑	↑	↑
Low	Mid	High

Is $15 == A[1]$? No

$15 > A[1]$, repeat the steps with $low = mid + 1 = 2$ and $high = 2$

Is $low > high$? NO

$$Mid = (2 + 2) / 2 = 2.$$

8	10	15
[0]	[1]	[2]
		↑
		Mid=high=2

Is $15 == A[2]$? Yes

Return (2).

Element 15 is present in the position 2.

Let us search for an element that is not in the list. Search Element=9

Is $low > high$? NO

$$Mid = (0 + 6) / 2 = 3.$$

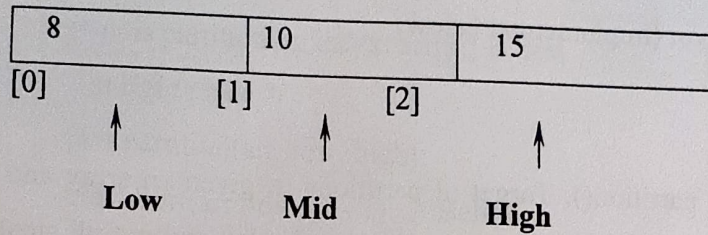
8	10	15	20	28	30	33
[0]	[1]	[2]	[3]	[4]	[5]	[6]
↑			↑			↑
low			mid			high

Is $9 == A[3]$? No

$9 < A[3]$, repeat the steps with $low = 0$ and $high = mid - 1 = 2$

Is $low > high$? NO

$Mid = (0 + 2) / 2 = 1$.

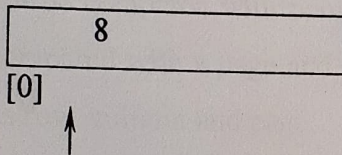


Is $9 == A[1]$? No

$9 < A[1]$, repeat the steps with $low = 0$ and $high = mid - 1 = 0$

Is $low > high$? NO

$Mid = (0 + 0) / 2 = 0$.



Low = mid = high = 0

Is $9 == A[0]$? No

Repeat the steps with $low = mid + 1 = 1$ and $high = 0$

Is $low > high$? Yes

When low value becomes greater than high value algorithm returns that the element 9 is not present in the list.

TIME COMPLEXITY:	BEST	WORST	AVERAGE
	O (1)	O(log n)	O(log n)

2. Explain the Quick sort algorithm (Nov 2017).

Quick Sort is a Divide and Conquer algorithm. It picks an element as pivot and partitions the given array around the picked pivot. There are many different versions of quickSort that pick pivot in different ways.

1. Always pick first element as pivot.

2. Always pick last element as pivot (implemented below)

Pick a random element as pivot.

Pick median as pivot.

The key process in quickSort is partition(). Target of partitions is, given an array and an element x of array as pivot, put x at its correct position in sorted array and put all smaller elements (smaller than x) before x, and put all greater elements (greater than x) after x. All this should be done in linear time.

Pseudo Code for recursive QuickSort function :

/* low --> Starting index, high --> Ending index */

quickSort(arr[], low, high)

```
{
    if (low < high)
    {
        /* pi is partitioning index, arr[pi] is now
           at right place */
        pi = partition(arr, low, high);
```

```
        quickSort(arr, low, pi - 1); // Before pi
```

```
        quickSort(arr, pi + 1, high); // After pi
```

```
    }
```

```
}
```

quicksort

Partition Algorithm

There can be many ways to do partition, following pseudo code adopts the method given in CLRS book. The logic is simple, we start from the leftmost element and keep track of index of smaller (or equal to) elements as i. While traversing, if we find a smaller element, we swap current element with arr[i]. Otherwise we ignore current element.


```
/* low --> Starting index, high --> Ending index */
quickSort(arr[], low, high)
{
    if (low < high)
    {
        /* pi is partitioning index, arr[pi] is now
        at right place */
        pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1); // Before pi
        quickSort(arr, pi + 1, high); // After pi
    }
}
```

3. Explain about Multi dimensional array (Nov 2019)

• If three or more subscript or index is required to refer the elements of an array then the array is called as three dimensional or multidimensional arrays. Three dimensional arrays can be compared with books whereas two-dimensional and one-dimensional arrays can be compared with a page and a line respectively. Here three major dimensions can be termed as row, column and page.

1. Number of rows = X (number of elements in a column)
2. Number of columns = Y (number of elements in a row)
3. Number of pages = Z

DECLARATION OF AN ARRAY:

The declaration can be as follows:

Example Program based on 3-D Arrays:

```
void main()
{
    int mark[2][4][3], i, j, k;
    int totmark, semsum, papsum;
    clrscr();
    for(i=0; i<2; i++)
    {
        printf("For the %d semester \n", i+1);
        for(j=0; j<4; j++)
```



```
        {   printf("\n Enter 3 marks for paper %d\n",j+1);
            for(k=0;k<3;k++)
                scanf("%d",&mark[i][j][k]);
        }
    }   totmark=0;
    for(i=0;i<2;i++)
    {   semsum=0;
        printf("\n Semester %d\n",i+1);
        printf("Paper marks \n");
        for(j=0;j<4;j++)
        {   papsum=0;
            for(k=0;k<3;k++)
                papsum+=mark[i][j][k];
            printf("%d   %d\n",j+1,papsum);
            semsum+=papsum;}
        printf("Total marks in %d semester = %d \n",i+1,semsum);
        totmark+=semsum;}
    printf("\n Grand total = %d",totmark);
    getch();
```

4. Explain in detail about Merge Sort (Nov 2017)

Merge Sort is a Divide and Conquer algorithm. It divides the input array into two halves, calls itself for the two halves, and then merges the two sorted halves. The merge() function is used for merging two halves. The merge(arr, l, m, r) is a key process that assumes that arr[l..m] and arr[m+1..r] are sorted and merges the two sorted sub-arrays into one. See the following C implementation for details.

MergeSort(arr[], l, r)

If $r > 1$

1. Find the middle point to divide the array into two halves:

$$\text{middle } m = l + (r-1)/2$$

2. Call mergeSort for first half:

Call mergeSort(arr, l, m)

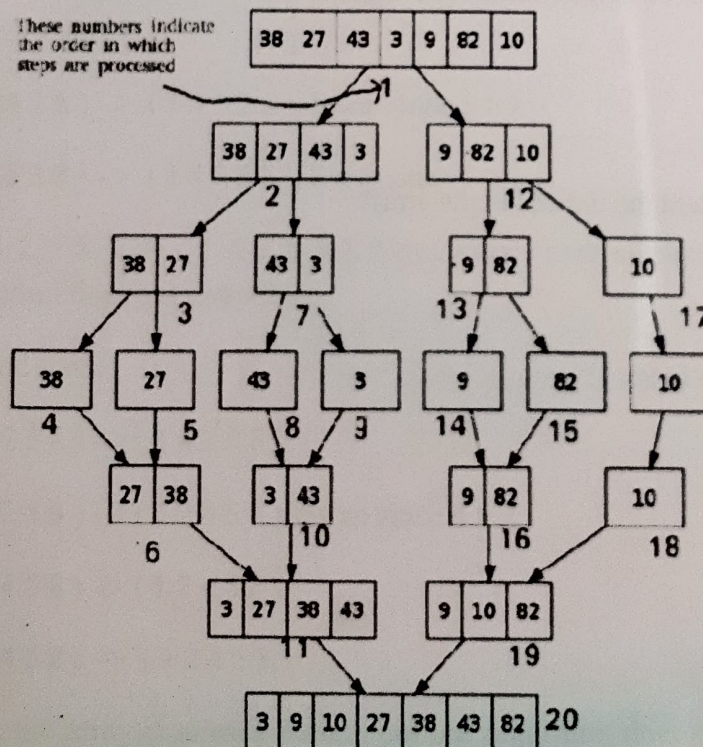
3. Call mergeSort for second half:

Call mergeSort(arr, m+1, r)

4. Merge the two halves sorted in step 2 and 3:

Call merge (arr, l, m, r)

The following diagram shows the complete merge sort process for an example array {38, 27, 43, 3, 9, 82, 10}. If we take a closer look at the diagram, we can see that the array is recursively divided in two halves till the size becomes 1. Once the size becomes 1, the merge processes come into action and start merging arrays back till the complete array is merged.



5. What is heap sort? Explain with an Example (Dec 2016)

Heap sort is a comparison based sorting technique based on Binary Heap data structure. It is similar to selection sort where we first find the maximum element and place the maximum element at the end. We repeat the same process for the remaining elements.

Heap Sort Algorithm for sorting in increasing order:

1. Build a max heap from the input data.
2. At this point, the largest item is stored at the root of the heap. Replace it with the last item of the heap followed by reducing the size of heap by 1. Finally, heapify the root of the tree.
3. Repeat step 2 while size of heap is greater than

Input data: 4, 10, 3, 5, 1

4(0)

/ \

10(1) 3(2)

/ \

5(3) 1(4)

The numbers in bracket represent the indices in the array representation of data.

Applying heapify procedure to index 1:

4(0)

/ \

10(1) 3(2)

/ \

5(3) 1(4)

Applying heapify procedure to index 0:

10(0)

/ \

5(1) 3(2)

/ \

4(3) 1(4)

The heapify procedure calls itself recursively to build heap in top down manner.

6. Explain Bubble sort and Quick sort with example (April 2017)

Bubble Sort is the simplest sorting algorithm that works by repeatedly swapping the adjacent elements if they are in wrong order.

Example:

First Pass:

(5 1 4 2 8) $\rightarrow$ (1 5 4 2 8), Here, algorithm compares the first two elements, and swaps since $5 > 1$.

(1 5 4 2 8) $\rightarrow$ (1 4 5 2 8), Swap since $5 > 4$

(1 4 5 2 8) $\rightarrow$ (1 4 2 5 8), Swap since $5 > 2$

(1 4 2 5 8) $\rightarrow$ (1 4 2 5 8), Now, since these elements are already in order ($8 > 5$), algorithm does not swap them.

Second Pass:

(1 4 2 5 8) $\rightarrow$ (1 4 2 5 8)

(1 4 2 5 8) $\rightarrow$ (1 2 4 5 8), Swap since $4 > 2$

(1 2 4 5 8) $\rightarrow$ (1 2 4 5 8)

(1 2 4 5 8) $\rightarrow$ (1 2 4 5 8)

Now, the array is already sorted, but our algorithm does not know if it is completed. The algorithm needs one whole pass without any swap to know it is sorted.

Third Pass:

$(1\ 2\ 4\ 5\ 8) \rightarrow (1\ 2\ 4\ 5\ 8)$

$(1\ 2\ 4\ 5\ 8) \rightarrow (1\ 2\ 4\ 5\ 8)$

$(1\ 2\ 4\ 5\ 8) \rightarrow (1\ 2\ 4\ 5\ 8)$

$(1\ 2\ 4\ 5\ 8) \rightarrow (1\ 2\ 4\ 5\ 8)$

QuickSort is a Divide and Conquer algorithm. It picks an element as pivot and partitions the given array around the picked pivot. There are many different versions of quickSort that pick pivot in different ways.

1. Always pick first element as pivot.
2. Always pick last element as pivot (implemented below)
3. Pick a random element as pivot.
4. Pick median as pivot.

The key process in quickSort is partition(). Target of partitions is, given an array and an element x of array as pivot, put x at its correct position in sorted array and put all smaller elements (smaller than x) before x , and put all greater elements (greater than x) after x . All this should be done in linear time.

7. Explain the Asymptotic notation (April 2017)

ASYMPTOTIC NOTATIONS

Complexity of an algorithm is commonly represented by following asymptotic notations,

1. $\Theta(x)$: Means that the rate varies directly with that of the constant x .
2. $\Theta(n)$: Means that the rate varies directly with size of the input.
3. $\Theta(n^2)$: Means that the rate varies with the square of the size of the input.
4. $\Theta(n^3)$: Means that the rate varies with the cube of the size of the input.
5. $\Theta(n!)$: Means that the rate varies with the factorial of the size of the input.
6. $\Theta(2^n)$: Means that the rate varies exponentially with the size of the input.
7. $\Theta(\log n)$: Means that the rate varies with the logarithm (power to which 2 has to be raised to get n) of the size of the input.
8. $\Theta(n \log n)$: Means that the rate varies n times with the logarithm of the size of the input.

The following notations are the methods used to estimate the efficiency of an algorithm:

1. Big-Oh Notation(O)
2. Big-Omega Notation(Ω)

3. Big-Theta Notation(θ)
4. Little-oh Notation(o)
5. Little -omega Notation(ω)

8. State the Insertion sort with an Example. (Nov 2015)

Insertion sort is a simple sorting algorithm that works similar to the way you sort playing cards in your hands. The array is virtually split into a sorted and an unsorted part. Values from the unsorted part are picked and placed at the correct position in the sorted part.

Algorithm

To sort an array of size n in ascending order:

- 1: Iterate from $arr[1]$ to $arr[n]$ over the array.
- 2: Compare the current element (key) to its predecessor.
- 3: If the key element is smaller than its predecessor, compare it to the elements before. Move the greater elements one position up to make space for the swapped element.

Example:

Insertion-sort

Another Example:

12, 11, 13, 5, 6

Let us loop for $i = 1$ (second element of the array) to 4 (last element of the array)

$i = 1$. Since 11 is smaller than 12, move 12 and insert 11 before 12

11, 12, 13, 5, 6

$i = 2$. 13 will remain at its position as all elements in $A[0..i-1]$ are smaller than 13

11, 12, 13, 5, 6

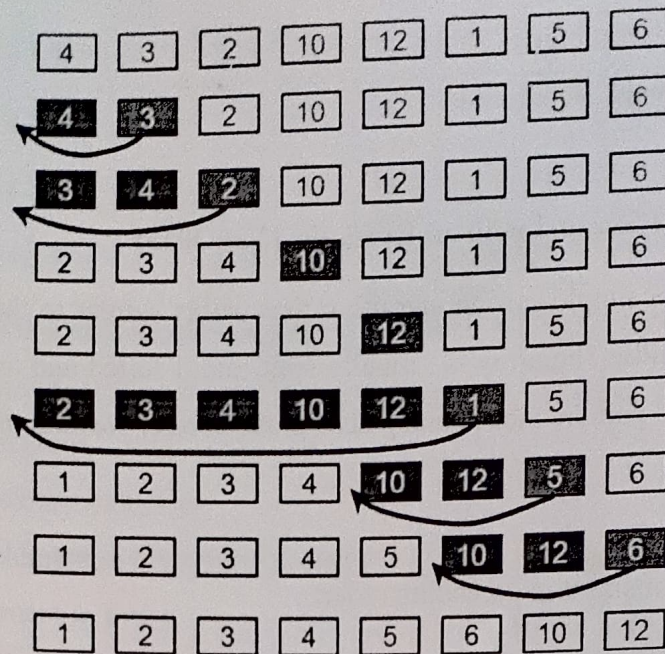
$i = 3$. 5 will move to the beginning and all other elements from 11 to 13 will move one position ahead of their current position.

5, 11, 12, 13, 6

$i = 4$. 6 will move to position after 5, and elements from 11 to 13 will move one position ahead of their current position.

5, 6, 11, 12, 13

Insertion Sort Execution Example



9. What is Heap sort? Give an algorithm for Heap sort. What is its time complexity? 42, 23, 75, 12, 66, 35, 96, 58, 9, 82. (Sep 2020)

Heap sort is a comparison based sorting technique based on Binary Heap data structure. It is similar to selection sort where we first find the maximum element and place the maximum element at the end. We repeat the same process for the remaining elements.

Let us first define a Complete Binary Tree. A complete binary tree is a binary tree in which every level, except possibly the last, is completely filled, and all nodes are as far left as possible (Source Wikipedia)

A Binary Heap is a Complete Binary Tree where items are stored in a special order such that value in a parent node is greater (or smaller) than the values in its two children nodes. The former is called as max heap and the latter is called min-heap. The heap can be represented by a binary tree or array.

Heap Sort has $O(n \log n)$ time complexities for all the cases (best case, average case, and worst case). ... Also it performs sorting in $O(1)$ space complexity.

